High-Quality Sender Diagnostics with Constexpr Exceptions

Document #:	P3557R1
Date:	2025-02-13
Project:	Programming Language C++
Audience:	LEWG Library Evolution
Reply-to:	Eric Niebler <eric.niebler@gmail.com>

Contents

"Only the exceptional paths bring exceptional glories!"
— Mehmet Murat Ildan

§ 1 Introduction

The hardest part of writing a sender algorithm is often the computation of its completion signatures, an intricate meta-programming task. Using sender algorithms incorrectly leads to large, incomprehensible errors deep within the completion-signatures meta-program. What is needed is a way to propagate type errors automatically to the API boundary where they can be reported concisely, much the way exceptions do for runtime errors.

Support for exceptions during constant-evaluation was recently accepted into the Working Draft for C++26. We can take advantage of this powerful new feature to easily propagate type errors during the computation of a sender's completion signatures. This significantly improves the diagnostics users are likely to encounter while also simplifying the job of writing new sender algorithms.

§ 2 Executive Summary

This paper proposes the following changes to the working draft with the addition of [P3164R3]. Subsequent sections will address the motivation and the designs in detail.

1. Change `std::execution::get_completion_signatures` from a customization point object that accepts a sender and (optionally) an environment to a `consteval` function template that takes no arguments, as follows:

<table>
<tr><th>Before</th><th>After</th></tr>
<tr><td>

```
inline constexpr struct
        get_completion_signatures_t
        {
  template <class Sndr, class...
        Env>
  auto operator()(Sndr&&, Env&&...)
        const -> see below;
} get_completion_signatures {};
```

</td><td>

```
template <class Sndr, class... Env>
consteval auto
        get_completion_signatures()
  -> valid-completion-signatures
        auto;
```

</td></tr>
</table>

2. Change the mechanism by which senders customize `get_completion_signatures` from a member function that accepts the *cv*-qualified sender object and an optional environment object to a `static constexpr` function template that take the sender and environment types as template parameters.

Before	After

<pre>struct my_sender {
  template <class Self, class... Env>
    requires some-predicate<Self,
        Env...>
  auto get_completion_signatures(this
        Self&&, Env&&) {
    return completion_signatures</* …
        */>();
  }
  ...
};</pre>
	<pre>struct my_sender {
	  template <class Self, class...
	        Env>
	  static constexpr auto
	        get_completion_signatures()
	        {
	    if constexpr (!some-
	        predicate<Self, Env...>) {
	      throw a-helpful-diagnostic();
	        // <--- LOOK!
	    }
	    return completion_signatures</*
	        … */>();
	  }
	  ...
	};</pre>

3. Change the `sender_in<Sender, Env...>` concept to test that `get_completion_signatures<Sndr, Env...>()` is a constant expression.

Before	After

<pre>template<class Sndr, class...
        Env>
concept sender_in =
  sender<Sndr> &&
  (queryable<Env> &&...) &&
  requires (Sndr&& sndr,
        Env&&... env) {
    { get_completion_signatures(
        std::forward<Sndr>
        (sndr),
        std::forward<Env>
        (env)...) }
          -> valid-completion-
        signatures;
  };</pre>
	<pre>template <auto>
	concept is-constant = true; // exposition only
	
	template<class Sndr, class... Env>
	concept sender_in =
	  sender<Sndr> &&
	  (queryable<Env> &&...) &&
	  is-constant<get_completion_signatures<Sndr,
	      Env...>()>;</pre>

4. In the exposition-only *`basic-sender`* class template, specify under what conditions its `get_completion_signatures` static member function is ill-formed when called without an `Env` template parameter (see proposed wording for details).

5. Add a `dependent_sender` concept that is modeled by sender types that do not know how they will complete independent of their execution environment.

6. [Optional]: Remove the `transform_completion_signatures` alias template.

The following additions are suggested by this paper to make working with completion signatures in `constexpr` code easier. None of these additions is strictly necessary.

— Extend the `completion_signatures` class template with `constexpr` operations that make the manipulation of completion signatures more ergonomic. These extensions are:

- Combining two sets of completion signatures with `operator+`. Alternative: provide a variadic `concat_completion_signatures` function.

- Treating an instance of a completion signatures specialization as a tuple of function pointers. For example, `completion_signatures<set_value_t(int), set_stopped_t()>{}` would be usable as if it were `tuple<set_value_t(*)(), set_stopped_t(*)()>{nullptr, nullptr}`, making `std::apply` useful for manipulating completion signatures.

- Adding CTAD to `completion_signatures` to deduce the signature types from a list of function pointers. Together with the above item, the following is the identity transform, given a `completion_signatures` object `cs`:

```
auto cs2 = std::apply([](auto... sigs){ return
    completion_signatures{sigs...}; }, cs);
static_assert(std::same_as<decltype(cs), decltype(cs2)>);
```

— Add a `make_completion_signatures` helper function that takes signatures as template arguments or as function arguments or both. The returned value would have signatures that have been normalized, sorted ([P2830R7]), and made unique.

— Add a `get_child_completion_signatures` function template that makes it easy for a sender adaptor to get the completion signatures of the child sender with the proper *cv* qualification. It is simply:

```
template <class Parent, class Child, class... Env>
consteval auto get_child_completion_signatures() {
  using CvChild = decltype(std::forward_like<Parent>
    (declval<Child&>()));
  return get_completion_signatures<CvChild, FWD-ENV(Env)...>();
}
```

— Reintroduce `transform_completion_signatures` as a `constexpr` function template that accepts lambdas as transforms. For example, the following code removes all error completions from a set, `cs`:

```
auto cs2 = transform_completion_signatures(
  cs,
  {}, // accept the default value completion transform
  []<class Error>() { return completion_signatures(); });
```

See `transform_completion_signatures` for a design description and reference implementation of the `transform_completion_signatures` function template.

— Add an `invalid_completion_signature` `consteval` function template whose return type is `completion_signatures<>` but that throws an unspecified exception type that contains diagnostic information. The `read_env` sender might use it as follows:

```
template <class Q>
template <class Self, class Env>
static constexpr auto
    read_env_sender<Q>::get_completion_signatures() {
  namespace exec = std::execution;
  if constexpr (!std::invocable<Q, Env>) {
```

```
      // return type deduced to be completion_signatures<> but
         function exits
     // with an exception that contains the relevant diagnostic
         information.
    return exec::invalid_completion_signature<
      _IN_ALGORITHM<read_env>,
      _WITH_QUERY(Q),
      _WITH_ENVIRONMENT(Env)
    >("The environment does not provide a value for the given
         query type.");
  } else {
    using Result = std::invoke_result_t<Q, Env>;
                                                           return
         exec::completion_signatures<exec::set_value_t(Result)>
         ();
  }
}
```

§ 3 Revision History

§ 3.1 R1

Since R0, a significant fraction of C++26's `std::execution` has been implemented with the design changes proposed by this paper. Several bugs in R0 have been found and fixed as a result.

In addition, this paper exposed several bugs in the Working Draft for C++26. As those bugs relate to the computation of completion signatures, R1 integrates the proposed fixes for those bugs.

- Fix the specification of `std::execution::get_completion_signatures` which was the victim of an incomplete last-minute edit.
- Add support for awaitables, which was dropped accidentally, back to `get_completion_signatures`.
- Change proposed replacement for `transform_completion_signatures` to take a transform function for the stopped completion signature instead of a `completion_signatures` object.
- Add missing semantic constraints to *`impls-for`*`<Tag>::`*`check-types`* customizations.
- Add missing syntactic constraint on *`basic-sender`*`<Tag>::get_completion_signatures`.
- Strengthen the *Mandates* on `std::execution::connect`.
- Fixed the constraint on `stopped_as_optional.transform_sender(sndr, env)`.
- Report type errors from the `let_*`, `schedule_from`, `into_variant`, `split`, and `when_all` algorithms when the preceessor's result datums cannot be copied into intermediate storage.
- Add proposed wording for the nice-to-have features:
  - `eptr_completion_if`
  - `make_completion_signatures`

  - `transform_completion_signatures` (a `constexpr` function version)
  - `invalid_completion_signature`
- Integrate proposed fix for cplusplus/sender-receiver#311, "`stopped_as_optional` tests constraints satisfaction of self instead of child".
- Integrate proposed fix for cplusplus/sender-receiver#317, "Several algorithms are not applying *`FWD-ENV`* as they should".
- Integrate proposed fix for cplusplus/sender-receiver#318, "specification of `as-sndr2(Sig)` in [exec.let] is incomplete".
- Integrate proposed fix for cplusplus/sender-receiver#319, "`let_[*].transform_env` is specified in terms of the `let_*` sender itself instead of its child".
- Fix paragraph numbering.

§ 3.2 R0

- Initial revision

§ 4 Motivation

This paper exists principly to improve the experience of users who make type errors in their sender expressions by leveraging exceptions during constant- evaluation. It is a follow-on of [P3164R2], which defines a category of "non-dependent" senders that can and must be type-checked early.

Senders have a construction phase and a subsequent connection phase. Prior to P3164, all type-checking of senders happened at the connection phase (when a sender is connected to a receiver). P3164 mandates that the sender algorithms type-check non-dependent senders, moving the diagnostic closer to the source of the error.

This paper addresses the *quality* of those diagnostics and the diagnostics users encounter when a dependent sender fails type-checking at connection time.

Senders are expression trees, and type errors can happen deep within their structure. If programmed naively, ill-formed senders would generate megabytes of incomprehensible diagnostics. The challenge is to report type errors *concisely* and *comprehensibly*, at the right level of abstraction.

Doing this requires propagating domain-specific descriptions of type errors out of the completion signatures meta-program so they can be reported concisely. Such error detection and propagation is very cumbersome in template meta-programming.

The C++ solution to error propagation is exceptions. With the adoption of [P3068R6], C++26 has gained the ability to throw and catch exceptions during constant-evaluation. If we express the computation of completion signatures as a `constexpr` meta-program, we can use exceptions to propagate type errors. This greatly improves diagnostics and even simplifies the code that computes completion signatures.

This paper proposes changes to `std::execution` that make the computation of a sender's completion signatures an evaluation of a `constexpr` function. It also specifies the conditions under which the computation is to exit with an exception.

§ 5 Proposed Design, Necessary Changes

§ 5.1 `get_completion_signatures`

In the Working Draft, a sender's completion signatures are determined by the type of the expression `std::execution::get_completion_signatures(sndr, env)` (or, after P3164, `std::execution::get_completion_signatures(sndr)` for non-dependent senders). Only the type of the expression matters; the expression itself is never evaluated.

In the design proposed by this paper, the `get_completion_signatures` expression must be constant-evaluated in order use exceptions to report errors. To make it ammenable to constant evaluation, it must not accept arguments with runtime values, so the expression is changed to `std::execution::get_completion_signatures<Sndr, Env...>()`, where `get_completion_signatures` is a `consteval` function.

If an unhandled exception propagates out of `get_completion_signatures` the program is ill-formed (because `get_completion_signatures` is `consteval`). The diagnostic displays the type and value of the exception.

`std::execution::get_completion_signatures<Sndr, Env...>()` in turn calls `remove_reference_t<Sndr>::template get_completion_signatures<Sndr, Env...>()`, which computes the completion signatures or throws as appropriate, as shown below:

```
namespace exec = std::execution;

struct void_sender {
  using sender_concept = exec::sender_t;

  template <class Self, class... Env>
  static constexpr auto get_completion_signatures() {
    return exec::completion_signatures<exec::set_value_t()>();
  }

  /* … more … */
};
```

To better support the `constexpr` value-oriented programming style, calls to `get_completion_signatures` from a `constexpr` function are never ill-formed, and they always have a `completion_signatures` type. `get_completion_signatures` reports errors by failing to be a constant expression.

§ 5.1.1 Non-non-dependent senders

[P3164R3] introduces the concept of non-dependent senders: senders that have the same completion signatures regardless of the receiver's execution environment. For a sender type

`DependentSndr` whose completions *do* depend on the environment, what should happen when the sender's completions are queried without an environment? That is, what should the semantics be for `get_completion_signatures<DependentSndr>()`?

`get_completion_signatures<DependentSndr>()` should follow the general rule: it should be well-formed in a `constexpr` function, and it should have a `completion_signatures` type. That way, sender adaptors do not need to do anything special when computing the completions of child senders that are dependent. So `get_completion_signatures<DependentSndr>()` should throw.

If `get_completion_signatures<Sndr>()` throws for dependent senders, and it also throws for non-dependent senders that fail to type-check, how then do we distinguish between valid dependent and invalid non-dependent senders? We can distinguish by checking the type of the exception.

An example will help. Consider the `read_env(q)` sender, a dependent sender that sends the result of calling `q` with the receiver's environment. It cannot compute its completion signatures without an environment. The natural way for the `read_env` sender to express that is to require an `Env` parameter to its customization of `get_completion_signatures`:

```
namespace exec = std::execution;

template <class Query>
struct read_env_sender {
  using sender_concept = exec::sender_t;

  template <class Self, class Env> // NOTE: Env is not optional!
  static constexpr auto get_completion_signatures() {
    if constexpr (!std::invocable<Query, Env>) {
      throw exception-type-goes-here();
    } else {
      using Result = std::invoke_result_t<Query, Env>;
       return exec::completion_signatures<exec::set_value_t(Result)>
         ();
    }
  }

  /* … more … */
};
```

That makes `read_env_sender<Q>::get_completion_signatures<Sndr>()` an ill-formed expression, which the `get_completion_signatures` function can detect. In such cases, it would throw an exception of a special type that it can catch later when distinguishing between dependent and non-dependent senders.

§ 5.1.2 Implementation

Since the design has several parts, reading the implementation of `get_completion_signatures` is probably the easiest way to understand it. The implementation is shown below with comments describing the parts.

```
// This macro expands to an invocation of SNDR's get_completion_signatures
// customization.
#define GET_COMPLSIGS(SNDR, ...) std::remove_reference_t<SNDR>::template \
    get_completion_signatures<SNDR __VA_OPT__(,) __VA_ARGS__>()

// This macro expands to an evaluation of EXPR, followed by an invocation
// of the _checked_complsigs function which validates its type.
#define CHECK_COMPLSIGS(EXPR) (EXPR, _check_complsigs<decltype(EXPR)>())

template <class Sndr>
using _nested_complsigs_t = std::remove_reference_t<Sndr>::completion_signatures;

// This helper ensures that the passed type is indeed a specialization of the
// completion_signatures class template, and throws if it is not.
template <class Completions>
consteval auto _check_complsigs() {
  if constexpr (_valid_completion_signatures<Completions>)
    // We got a type that is a specialization of the completion_signatures
    // class template representing the sender's completions. Return it.
    return Completions();
  else
    // invalid_completion_signature throws unconditionally. Its return type
    // is `completion_signatures<>`, which prevents downstream errors that
    // would occur in other customizations of `get_completion_signatures`
    // if computing the completions of a child returned an object with a
    // surprising type.
    return invalid_completion_signature<unspecified>(unspecified);
}

template <class Sndr, class... Env>
consteval auto _get_completion_signatures_helper() {
  // The following `if` tests whether GET_COMPLSIGS(Sndr, Env...)
  // is a well-formed expression.
  if constexpr (requires { GET_COMPLSIGS(Sndr, Env...); }) {
    // The GET_COMPLSIGS(Sndr, Env...) expression is well-formed, but it may
    // throw an exception or otherwise fail to be a constant expression.
    // By evaluating it, we cause its exception and its non-constexpr-ness to
    // propagate. Then CHECK_COMPLSIGS ensures that its type is indeed
    // a specialization of the completion_signatures class template.
    return CHECK_COMPLSIGS(GET_COMPLSIGS(Sndr, Env...));
  }
  // The following `if` does the same as above, but for GET_COMPLSIGS(Sndr).
```

```
  // A non-dependent sender may announce itself by way of this signature.
  else if constexpr (requires { GET_COMPLSIGS(Sndr); }) {
    // Same as above: propagate any exceptions and non-constexpr-ness, and
    // verify the expression has the right type.
    return CHECK_COMPLSIGS(GET_COMPLSIGS(Sndr));
  }
  // Test whether Sndr has a nested ::completion_signatures type alias:
  else if constexpr (requires { _nested_complsigs_t<Sndr>(); }) {
    // It has the nested type alias, but does it denote a specialization of
    // the completion_signatures class template?
    return CHECK_COMPLSIGS(_nested_complsigs_t<Sndr>());
  }
  // If none of the above expressions are well-formed, then we don't know
  // the sender's completions. If we are testing without an environment, then
  // we assume Sndr is a dependent sender. Throw an exception that
  // communicates that.
  else if constexpr (sizeof...(Env) == 0) {
    return (throw dependent-sender-error(), completion_signatures());
  }
  else {
    // We cannot compute the completion signatures for this sender and
    // environment. Give up and throw an exception.
    return invalid_completion_signature<unspecified>(unspecified);
  }
}

template <class Sndr>
consteval auto get_completion_signatures() -> _valid_completion_signatures auto {
  // There is no environment, which means we are asking for the sender's non-
  // dependent completion signatures. If the sender is dependent, this will
  // exit with a special exception type.
  return _get_completion_signatures_helper<Sndr>();
}

template <class Sndr, class Env>
consteval auto get_completion_signatures() -> _valid_completion_signatures auto {
  // Apply a lazy sender transform if one exists before computing the completion signatures:
  using Domain = decltype(_get_domain_late(std::declval<Sndr>(), std::declval<Env>()));
  using NewSndr = decltype(transform_sender(Domain(), std::declval<Sndr>(), std::declval<Env>()));

  return _get_completion_signatures_helper<NewSndr, Env>();
}
```

Given this definition of `get_completion_signatures`, we can implement a `dependent_sender` concept as follows:

```
// Returns true when get_completion_signatures<Sndr>() throws a
// dependent-sender-error. Returns false when
// get_completion_signatures<Sndr>() returns normally (Sndr is non-
        dependent),
// or when it throws any other kind of exception (Sndr fails type-
        checking).
template <class Sndr>
consteval bool is-dependent-sender-helper() try {
  (void) get_completion_signatures<Sndr>();
  return false;
} catch (dependent-sender-error&) {
  return true;
}

template <class Sndr>
concept dependent_sender =
       sender<Sndr>    &&    std::bool_constant<is-dependent-sender-
        helper<Sndr>()>::value;
```

After the adoption of [P3164R3], the sender algorithms are all required to return senders that are either dependent or else that type-check successfully. This paper proposes adding that type-checking as a *Mandates* on the exposition-only *`make-sender`* function template that all the algorithms use to construct their return value.

Users who define their own sender algorithms can use `dependent_sender` and `get_completion_signatures` to perform early type-checking of their own sender types using a helper such as the following:

```
template <class Sndr>
constexpr auto _type_check_sender(Sndr sndr) {
  if constexpr (!dependent_sender<Sndr>) {
      // This line will fail to compile if Sndr fails its type
        checking. We
      // don't want to perform this type checking when Sndr is
        dependent, though.
       // Without an environment, the sender doesn't know its
        completions.
    (void) get_completion_signatures<Sndr>();
  }
  return sndr;
}
```

Using this helper, a `then` algorithm might type-check its returned senders as follows:

```
inline constexpr struct then_t {
  template <sender Sndr, class Fn>
  auto operator()(Sndr sndr, Fn fn) const {
           return   _type_check_sender(_then_sender{std::move(sndr),
        std::move(fn)});
  }
} then {};
```

§ 5.2 `sender_in`

With the above changes, we need to tweak the `sender_in` concept to require that `get_completion_signatures<Sndr, Env...>()` is a constant expression.

The changes to `sender_in` relative to [P3164R3] are as follows:

```
template <auto>
  concept is-constant = true; // exposition only

template<class Sndr, class... Env>
  concept sender_in =
    sender<Sndr> &&
    (sizeof...(Env) <= 1)
    (queryable<Env> &&...) &&
    is-constant<get_completion_signatures<Sndr, Env...>()>;
    requires (Sndr&& sndr, Env&&... env) {
              { get_completion_signatures(std::forward<Sndr>(sndr),
        std::forward<Env>(env)...) }
        -> valid-completion-signatures;
    };
```

§ 5.3 *basic-sender*

The sender algorithms are expressed in terms of the exposition-only class template *basic-sender*. The mechanics of computing completion signatures is not specified, however, so very little change there is needed to implement this proposal.

We do, however, have to say when *basic-sender*`::get_completion_signatures<Sndr>()` is ill-formed. In [P3164R3], non-dependent senders are dealt with by discussing whether or not a sender's potentially-evaluated completion operations are dependent on the type of the receiver's environment. In this paper, we make a similar appeal when specifying whether or not *basic-sender*`::get_completion_signatures<Sndr>()` is well-formed.

§ 5.4 `dependent_sender`

Users who write their own sender adaptors will also want to perform early type-checking of senders that are not dependent. Therefore, they need a way to determine whether or not a sender is dependent.

In the section `get_completion_signatures` we show how the concept `dependent_sender` can be implemented in terms of this paper's `get_completion_signatures` function template. By making this a public-facing concept, we give sender adaptor authors a way to do early type-checking, just like the standard adaptors.

§ 6 Proposed Design, Nice-to-haves

§ 6.1 `completion_signatures`

Computing completions signatures is now to be done using `constexpr` meta-programming by manipulating values using ordinary imperative C++ rather than template meta-programming. To better support this style of programming, it is helpful to add `constexpr` operations that manipulate instances of specializations of the `completion_signatures` class template.

For example, it should be possible to take the union of two sets of completion signatures. `operator+` seems like a natural choice for that:

```
completion_signatures<set_value_t(int),  set_error_t(exception_ptr)>
        cs1;
completion_signatures<set_stopped_t(),   set_error_t(exception_ptr)>
        cs2;

auto cs3 = cs1 + cs2; // completion_signatures<set_value_t(int),
                                                                  //
        set_error_t(exception_ptr),
                     //                       set_stopped_t()>
```

`operator+` is nice because it is possible to do a variadic fold over a set of `completion_signatures` objects to concatenate them all:

```
[](auto... cs) { // a pack of completion_signatures objects
  return completion_signatures() +...+ cs;
}
```

A different option instead of `operator+` would be to use a free function that takes a variable number of `completion_signature` objects and concatenates all of them.

It can also be convenient for `completion_signatures` specializations to model *`tuple-like`*. Although tuple elements cannot have funtion type, they can have function *pointer* type. With this proposal, an object like `completion_signatures<set_value_t(int), set_stopped_t()>{}` behaves like `tuple<set_value_t(*)(int), set_stopped_t(*)()>{nullptr, nullptr}` (except that it wouldn't actually have to store the `nullptr`s). That would make it possible to manipulate completion signatures using `std::apply`:

```
auto cs = /* … */;

// Add an lvalue reference to all arguments of all signatures:
auto add_ref =       []<class T, class... As>(T(*)(As...)) -> T(*)
        (As&...) { return {}; };
auto     add_ref_all     =     [=](auto...     sigs)     {     return
        make_completion_signatures(add_ref(sigs)...); };

return std::apply(add_ref_all, cs);
```

The code above uses another nice-to-have feature: a `make_completion_signatures` helper function that deduces the signatures from the arguments, removes any duplicates, and returns a new instance of `completion_signatures`.

Consider trying to do all the above using template meta-programming. 😬

§ 6.2 `make_completion_signatures`

The `make_completion_signatures` helper function described just above would allow users to build a `completion_signatures` object from a bunch of signature types, or from function pointer objects, or a combination of both:

```
// Returns a default-initialized object of type
       completion_signatures<Sigs...>,
// where Sigs is the set union of the normalized ExplicitSigs and
       DeducedSigs.
template <completion-signature... ExplicitSigs, completion-
       signature... DeducedSigs>
constexpr auto make_completion_signatures(DeducedSigs*... sigs)
       noexcept
  -> valid-completion-signatures auto;
```

To "normalize" a completion signature means to strip rvalue references from the arguments. So, `set_value_t(int&&, float&)` becomes `set_value_t(int, float&)`. `make_completions_signatures` first normalizes all the signatures and then removes duplicates. ([P2830R7] lets us order types, so making the set unique will be $O(n \log n)$.)

§ 6.3 `transform_completion_signatures`

The current Working Draft has a utility to make type transformations of completion signature sets simpler: the alias template `transform_completion_signatures`. It looks like this:

```
template <class... As>
using value-transform-default =
       completion_signatures<set_value_t(As...)>;

template <class Error>
using error-transform-default =
       completion_signatures<set_error_t(Error)>;

template <valid-completion-signatures Completions,
            valid-completion-signatures OtherCompletions =
       completion_signatures<>,
          template <class...> class ValueTransform = value-
       transform-default,
         template <class> class ErrorTransform = error-transform-
       default,
            valid-completion-signatures StoppedCompletions =
       completion_signatures<set_stopped_t()>>
using transform_completion_signatures = /*see below*/;
```

Anything that can be done with `transform_completion_signatures` can be done in `constexpr` using `std::apply`, a lambda with `if constexpr`, and `operator+` of `completion_signatures` objects. In fact, we could even implement `transform_completion_signatures` itself that way:

```
template </* … as before … */>
using transform_completion_signatures =
  std::constant_wrapper< // see [@P2781R5]
    std::apply(
      [](auto... sigs) {
        return ([]<class T, class... As>(T (*)(As...)) {
          if constexpr (^^T == ^^set_value_t) { // use reflection to
        test type equality
            return ValueTransform<As...>();
          } else if constexpr (^^T == ^^set_error_t) {
            return ErrorTransform<As...[0]>();
          } else {
            return StoppedCompletions();
```

```
        }
      }(sigs) +...+ completion_signatures());
    },
    Completions()
  ) + OtherCompletions()
>::value_type;
```

This paper proposes dropping the `transform_completion_signatures` type alias since it is not in the ideal form for `constexpr` meta-programming, and since `std::apply` is good enough (sort of).

However, should we decide to keep the functionality of `transform_completion_signatures`, we can reexpress it as a `constexpr` function that accepts transforms as lambdas:

```
constexpr auto value-transform-default = []<class... As>() { return
        completion_signatures<set_value_t(As...)>(); };
constexpr auto error-transform-default = []<class Error>() { return
        completion_signatures<set_error_t(Error)>(); };
constexpr   auto   stopd-transform-default   =   []   {   return
        completion_signatures<set_stopped_t()>(); };

template <valid-completion-signatures Completions,
          class ValueTransform = decltype(value-transform-default),
          class ErrorTransform = decltype(error-transform-default),
                callable  StopdTransform  =  decltype(stopd-transform-
        default),
                    valid-completion-signatures  OtherCompletions  =
        completion_signatures<>>
consteval     auto     transform_completion_signatures(Completions
        completions,
                                                         ValueTransform
        value_transform = {},
                                                         ErrorTransform
        error_transform = {},
                                                         StopdTransform
        stopd_transform = {},
                                                       OtherCompletions
        other_completions = {})
  -> valid-completion-signatures auto;
```

The above form of `transform_completion_signatures` is more natural to use from within a `constexpr` function. It also makes it simple to accept the default for some arguments as shown below:

```
// Transform just the error completion signatures:
auto cs2 = transform_completion_signatures(cs, {}, []<class E>() {
        return /* … */; });
                                                    //   ^^  Accept the
        default value transform
```

Since accepting the default transforms is simple, we are able to move the infrequently used `OtherCompletions` argument to the end of the argument list.

A faithful mapping of the old alias template to the new function template would take a `completion_signatures` object for the stopped completions instead of a stopped transform function. After all, what is the point of a nullary callable that returns a `completion_signatures` object when you could just pass the object itself directly? The reason is because the stopped transform function might want to throw an exception. That is why the `transform_completion_signatures` function template takes 3 transform functions instead of two functions and a `completion_signatures` object.

Although the signature of this `transform_completion_signatures` function looks frightful, the implementation is quite straightforward, and seeing it might make it less scary:

```
// A concept that is modeled by lambdas like the following:
//   []<class... Ts>() { return completion_signatures<...>(); }
template <class Fn, class... As>
concept __meta_transform_with = requires (const Fn& fn) {
           {      fn.template      operator()<As...>()      }      ->
        __valid_completion_signatures;
};

template <class... As, class Fn>
consteval auto __apply_transform(const Fn& fn) {
  if constexpr (sizeof...(As) == 0)
    return fn();
  else if constexpr (__meta_transform_with<Fn, As...>)
    return fn.template operator()<As...>();
  else
    return invalid_completion_signature< … >( … ); // see below
}

template < /* … as shown above … */ >
consteval      auto      transform_completion_signatures(Completions
        completions,
                                                         ValueTransform
        value_transform,
                                                         ErrorTransform
        error_transform,
                                                         StopdTransform
        stopd_transform,
                                                       OtherCompletions
        other_completions) {
  auto transform1 = [=]<class T, class... As>(Tag(*)(As...)) {
       if  constexpr  (Tag()  ==  set_value)  //  see  "Completion  tag
        comparison" below
      return __apply_transform<As...>(value_transform);
    else if constexpr (Tag() == set_error)
      return __apply_transform<As...>(error_transform);
    else
      return __apply_transform<As...>(stopd_transform);
  };

  auto transform_all = [=](auto*... sigs) {
    return (transform1(sigs) +...+ completion_signatures());
  };

  return std::apply(transform_all, completions) + other_completions;
}
```

Like `get_completion_signatures`, `transform_completion_signatures` always returns a specialization of `completion_signatures` and reports errors by throwing exceptions. It expects the lambdas passed to it to do likewise (but handles it gracefully if they don't).

§ 6.4 invalid_completion_signature

The reason for the design change is to permit the reporting of type errors using exceptions. Let's look at an example where it would be desirable to throw an exception from `get_completion_signatures`: the `then` algorithm. We will use this example to motivate the rest of the design changes.

The `then` algorithm attaches a continuation to an async operation that executes when the operation completes successfully. With this proposal, a `then_sender`'s `get_completion_signatures` customization might be implemented as follows:

```
template <class Sndr, class Fun>
template <class Self, class... Env>
constexpr auto then_sender<Sndr, Fun>::get_completion_signatures() {
  // compute the completions of the (properly cv-qualified) child:
  using Child = decltype(std::forward_like<Self>(declval<Sndr&>()));
      auto   child_completions   =   get_completion_signatures<Child,
        decltype(FWD-ENV(declval<Env>()))...>();

   // This lambda is used to transform value completion signatures:
  auto value_transform = []<class... As>() {
    if constexpr (std::invocable<Fun, As...>) {
      using Result = std::invoke_result_t<Fun, As...>;
      return completion_signatures<set_value_t(Result)>();
    } else {
      // Oh no, the user made an error! Tell them about it.
      throw some-exception-object;
    }
  };

  // Transform just the value completions:
        return    transform_completion_signatures(child_completions,
        value_transform);
}
```

We would like to make it dead simple to throw an exception that will convey a domain-specific diagnostic to the user. That way, the authors of sender algorithms will be more likely to do so.

The `invalid_completion_signature` helper function is designed to make generating meaningful diagnostics easy. As an example, here is how the `then_sender`'s `completion_signatures` customization might use it:

```
template <const auto&> struct IN_ALGORITHM;

template <class Sndr, class Fun>
template <class Self, class... Env>
constexpr auto then_sender<Sndr, Fun>::get_completion_signatures() {
  /* … */
  // This lambda is used to transform value completion signatures:
  auto value_transform = []<class... As>() {
    if constexpr (std::invocable<Fun, As...>) {
      using Result = std::invoke_result_t<Fun, As...>;
      return completion_signatures<set_value_t(Result)>();
    } else {
      // Oh no, the user made an error! Tell them about it.
```

```
      return invalid_completion_signature<
        IN_ALGORITHM<std::execution::then>,
        struct WITH_FUNCTION(Fun),
        struct WITH_ARGUMENTS(As...)
      >("The function passed to std::execution::then is not callable
        "
        "with the values sent by the predecessor sender.");
    }
  };
  /* … */
}
```

When the user of `then` makes a mistake, say like with the expression “`just(42) | then([](){…})`”, they will get a helpful diagnostic like the following (relevant bits highlighted):

```
<source>:658:3: error: call to immediate function 'operator|
        <just_sender<int>>'
        is not a constant expression
  658 |   just(42      ) | then([](){})
      |   ^
<source>:564:14: note: 'operator|<just_sender<int>>' is an immediate
        function be
        cause its body contains a call to an immediate function
        '__type_check_sender<the
        n_sender<just_sender<int>, (lambda at <source>:658:19)>>'
        and that call is not a
        constant expression
    564 |             return    __type_check_sender(then_sender{{},
self.fn_, sndr});
                                                                     |
^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
<source>:358:11:    note:    unhandled    exception    of    type
        '__sender_type_check_failure<
        const char *, IN_ALGORITHM<then>, WITH_FUNCTION ((lambda at
        <source>:658:19)), W
        ITH_ARGUMENTS (int)>' with content {&"The function passed
        to std::execution::the
        n is not callable with the values sent by the predecessor
        sender."[0]} thrown fr
        om here
  358 |     throw    __sender_type_check_failure<Values...[0
], What...>(values...);
      |           ^
1 error generated.
Compiler returned: 1
```

The above is the *complete* diagnostic, regardless of how deeply nested the type error is. So long, megabytes of template spew!

Lambdas passed to `transform_completion_signatures` *should* return a `completion_signatures` specialization (although `transform_completion_signatures` recovers gracefully when they do not). The return type of `invalid_completion_signature` is `completion_signatures<>`. By “returning” the result of calling `invalid_completion_signature`, the deduced return type of the lambda is a `completion_signatures` type, as it should be.

A possible implementation of the invalid_completion_signature function is shown below:

```
template <class... What, class... Args>
struct sender-type-check-failure : std::exception { // exposition
        only
        constexpr sender-type-check-failure(Args... args) :
        args_{std::move(args)...} {}
   constexpr char const* what() const noexcept override { return
        unspecified; };
  std::tuple<Args...> args_; // exposition only
};

template <class... What, class... Args>
[[noreturn, nodiscard]]
consteval                                    completion_signatures<>
        invalid_completion_signature(Args... args) {
          throw sender-type-check-failure<What..., Args...>
        {std::move(args)...};
}
```

§ 6.5 get_child_completion_signatures

In the then_sender above, computing a child sender's completion signatures is a little awkward:

```
// compute the completions of the (properly cv-qualified) child:
using Child = decltype(std::forward_like<Self>(declval<Sndr&>()));
auto child_completions = get_completion_signatures<Child,
        decltype(FWD-ENV(declval<Env>()))...>();
```

Computing the completions of child senders will need to be done by every sender adaptor algorithm. We can make this simpler with a get_child_completion_signatures helper function:

```
// compute the completions of the (properly cv-qualified) child:
auto child_completions = get_child_completion_signatures<Self, Sndr,
        Env...>();
```

… where get_child_completion_signatures is defined as follows:

```
template <class Parent, class Child, class... Env>
consteval auto get_child_completion_signatures() {
     using cvref-child-type = decltype(std::forward_like<Parent>
        (declval<Child&>()));
   return get_completion_signatures<cvref-child-type, decltype(FWD-
        ENV(declval<Env>()))...>();
}
```

§ 6.6 Completion tag comparison

For convenience, we can make the completion tag types equality-comparable with each other. When writing sender adaptor algorithms, code like the following will be common:

```
[]<class Tag, class... Args>(Tag(*)(Args...)) {
  if constexpr (std::is_same_v<Tag, exec::set_value_t>) {
    // Do something
```

```
  }
  else {
    // Do something else
  }
}
```

Although certainly not hard, with reflection the tag type comparison becomes a litte simpler:

```
  if constexpr (^^Tag == ^^exec::set_value_t>) {
```

We can make this even easier by simply making the completion tag types equality-comparable, as follows:

```
  if constexpr (Tag() == exec::set_value) {
```

The author finds that this makes his code read better. Tag types would compare equal to themselves and not-equal to the other two tag types.

§ 6.7 eptr_completion_if

The following is a trivial utility that the author finds he uses surprisingly often. Frequently an async operation can complete exceptionally, but only under certain conditions. In cases such as those, it is necessary to add a `set_error_t(std::exception_ptr)` signature to the set of completions, but only when the condition is met.

This is made simpler with the following variable template:

```
template <bool PotentiallyThrowing>
inline constexpr auto eptr_completion_if =
  std::conditional_t<PotentiallyThrowing,

        completion_signatures<set_error_t(exception_ptr)>,
                     completion_signatures<>>();
```

Below is an example usage, from the `then` sender:

```
template <class Sndr, class Fun>
template <class Self, class... Env>
constexpr auto then_sender<Sndr, Fun>::get_completion_signatures() {
  auto cs = get_child_completion_signatures<Self, Sndr, Env...>();
   auto  value_fn  =  []<class...  As>()  {  /*  …  as  shown  in  section
        "invalid_completion_signature" */ };
   constexpr bool nothrow = /* … false if Fun can throw for any set
        of the predecessor's values */;

   // Use eptr_completion_if here as the "extra" set of completions
        that
   // will be added to the ones returned from the transforms.
     return  transform_completion_signatures(cs,  value_fn,  {},  {},
        eptr_completion_if<!nothrow>);
}
```

§ 7 Questions for LEWG

Assuming we want to change how completion signatures are computed as proposed in this paper, the author would appreciate LEWG's feedback about the suggested additions.

1. Do we want to use `operator+` to join two `completion_signatures` objects?
2. Do we want to make `completion_signatures<Sigs...>` *`tuple-like`* (where `completion_signatures<Sigs...>()` behaves like `tuple<Sigs*...>()`)?
3. Should we drop the `transform_completion_signatures` alias template?
4. Should we add a `make_completion_signatures` helper function that returns an instance of a `completion_signatures` type with its function types normalized and made unique?
5. Should we replace the `transform_completion_signatures` alias template with a `consteval` function that does the same thing but for values?
6. Do we want the `invalid_completion_signature` helper function to make it easy to generate good diagnostics when type-checking a sender fails.
7. Do we want the `get_child_completion_signatures` helper function to make is easy for sender adaptors to get a (properly *cv*-qualified) child sender's completion signatures?
8. Do we want to make the completion tag types (`set_value_t`, etc.) `constexpr` equality-comparable with each other?
9. Do we want the `eptr_completion_if` variable template, which is an object of type `completion_signatures<set_error_t(std::exception_ptr)>` or `completion_signatures<>` depending on a `bool` template parameter?

§ 8 Implementation Experience

A significant fraction of `std::execution` has been implemented with this design change. It can be found on Compiler Explorer[1] and in this GitHub gist[2]. This implementation includes all the design elements that would stress the completion signature computation including:

— Both dependent and non-dependent senders and sender adaptors, with tests for both early and late diagnosis of type errors.

— Transforming completion signatures with the proposed `transform_completion_signatures` function template

— The exposition-only *`basic-sender`* has been implemented with the changes `get_completion_signatures` customization syntax and the new *`impls-for`*`<Tag>::`*`check-types`* customization, which has been used to implement and type-check all the sender types, as specified.

— A sender that is implemented via a lowering to another sender type via a lazy `transform_sender` customization (`stopped_as_optional`).

— Using awaitables as senders and *vice versa*.

§ 9 Proposed Wording

[Editor's note: This wording is relative to the current working draft with the addition of [P3164R3]]

[Editor's note: Change [exec.general] as follows:]

? For function type `R(Args...)`, let *`NORMALIZE-SIG`*`(R(Args...))` denote the type `R(`*`remove-rvalue-reference-t`*`<Args>...)` where *`remove-rvalue-reference-t`* is an alias template that removes an rvalue reference from a type.

7 For function types `F1` and `F2` ~~denoting `R1(Args1...)` and `R2(Args2...)`, respectively,~~ *`MATCHING-SIG`*`(F1, F2)` is `true` if and only if `same_as<`~~`R1(Args1&&...), R2(Args2&&...)`~~*`NORMALIZE-SIG`*`(F1)`, *`NORMALIZE-SIG`*`(F2)>` is `true`.

8 For a subexpression `err`, let `Err` be `decltype((err))` and let *`AS-EXCEPT-PTR`*`(err)` be [Editor's note: … as before]

[Editor's note: Change [execution.syn] as follows:]

Header `<execution>` synopsis [execution.syn]

```
namespace std::execution {
  … as before …

  template<class Sndr, class... Env>
    concept sender_in = see below;

  template<class Sndr>
    concept dependent_sender = see below;

  template<class Sndr, class Rcvr>
    concept sender_to = see below;

  template<class... Ts>
    struct type-list;                                              // exposition only

  // [exec.getcomplsigs], completion signatures
  struct get_completion_signatures_t;
  inline constexpr get_completion_signatures_t get_completion_signatures {};

  [ Editor's note: This alias is moved below and modified. ]
  template<class Sndr, class... Env>
    requires sender_in<Sndr, Env...>
  using completion_signatures_of_t = call-result-t<get_completion_signatures_t, Sndr, Env...>;

  template<class... Ts>
    using decayed-tuple = tuple<decay_t<Ts>...>;                   // exposition only
```

```
… as before …

// [exec.util], sender and receiver utilities
// [exec.util.cmplsig] completion signatures
template<class Fn>
  concept completion-signature = see below;                    // exposition only

template<completion-signature... Fns>
  struct completion_signatures {};

template<class Sigs>
  concept valid-completion-signatures = see below;             // exposition only

struct dependent-sender-error : exception {};                  // exposition only

// [exec.getcomplsigs]
template<class Sndr, class... Env>
  consteval auto get_completion_signatures() -> valid-completion-signatures auto;

template<class Sndr, class... Env>
    requires sender_in<Sndr, Env...>
  using completion_signatures_of_t = decltype(get_completion_signatures<Sndr, Env...>());

// [exec.util.cmplsig.trans]
template<
  valid-completion-signatures InputSignatures,
  valid-completion-signatures AdditionalSignatures = completion_signatures<>,
  template<class...> class SetValue = see below,
  template<class> class SetError = see below,
  valid-completion-signatures SetStopped = completion_signatures<set_stopped_t()>>
using transform_completion_signatures = completion_signatures<see below>;

template<
  sender Sndr,
  class Env = env<>,
  valid-completion-signatures AdditionalSignatures = completion_signatures<>,
  template<class...> class SetValue = see below,
  template<class> class SetError = see below,
  valid-completion-signatures SetStopped = completion_signatures<set_stopped_t()>>
    requires sender_in<Sndr, Env>
using transform_completion_signatures_of =
  transform_completion_signatures<
    completion_signatures_of_t<Sndr, Env>,
    AdditionalSignatures, SetValue, SetError, SetStopped>;

// [exec.run.loop], run_loop
class run_loop;

… as before …
```

```
}
```

[Editor's note: Add the following paragraph after [execution.syn] para 3 (this is moved from [exec.snd.concepts])]

? A type models the exposition-only concept `*valid-completion-signatures*` if it denotes a specialization of the `completion_signatures` class template.

[Editor's note: Modify [exec.snd.general] as follows:]

1 Subclauses [exec.factories] and [exec.adapt] define customizable algorithms that return senders. Each algorithm has a default implementation. Let `sndr` be the result of an invocation of such an algorithm or an object equal to the result ([concepts.equality]), and let `Sndr` be `decltype((sndr))`. Let `rcvr` be a receiver of type `Rcvr` with associated environment `env` of type `Env` such that `sender_to<Sndr, Rcvr>` is `true`. For the default implementation of the algorithm that produced `sndr`, connecting `sndr` to `rcvr` and starting the resulting operation state ([exec.async.ops]) necessarily results in the potential evaluation ([basic.def.odr]) of a set of completion operations whose first argument is a subexpression equal to `rcvr`. Let `Sigs` be a pack of completion signatures corresponding to this set of completion operations, and let `CS` be the type of the expression ~~`get_completion_signatures(sndr, env)`~~ `get_completion_signatures<Sndr, Env>()`. Then `CS` is a specialization of the class template `completion_signatures` ([exec.util.cmplsig]), the set of whose template arguments is `Sigs`. If none of the types in `Sigs` are dependent on the type `Env`, then the expression ~~`get_completion_signatures(sndr)`~~ `get_completion_signatures<Sndr>()` is well-formed and its type is `CS`. If a user-provided implementation of the algorithm that produced `sndr` is selected instead of the default:

(1.1) — Any completion signature that is in the set of types denoted by `completion_signatures_of_t<Sndr, Env>` and that is not part of `Sigs` shall correspond to error or stopped completion operations, unless otherwise specified.

(1.2) — If none of the types in `Sigs` are dependent on the type `Env`, then `completion_signatures_of_t<Sndr>` and `completion_signatures_of_t<Sndr, Env>` shall denote the same type.

[Editor's note: In [exec.snd.expos], change para 2 as follows:]

2 For a queryable object `env`, `*FWD-ENV*(env)` is an expression whose type satisfies `queryable` such that for a query object `q` and a pack of subexpressions `as`, the expression `*FWD-ENV*(env).query(q, as...)` is ill-formed if `forwarding_query(q)` is `false`; otherwise, it is expression-equivalent to `env.query(q, as...)`. The type `*FWD-ENV-T*(Env)` is `decltype(*FWD-ENV*(declval<Env>()))`.

[Editor's note: In [exec.snd.expos], insert the following paragraph after para 22 and before para 23 (moving the exposition-only alias template out of para 24 and into its own para so it can be used from elsewhere):]

23 Let *valid-specialization* be the following alias template:

```
template<template<class...> class T, class... Args>
   concept  valid-specialization  =  requires  {  typename
        T<Args...>; }; // exposition only
```

[Editor's note: In [exec.snd.expos] para 23 add the mandate below, and in para 24, change the definition of the exposition-only *basic-sender* as follows:]

```
template<class Tag, class Data = see below, class... Child>
   constexpr  auto  make-sender(Tag  tag,  Data&&  data,  Child&&...
        child);
```

23 *Mandates*: The following expressions are `true`:

(23.1) — `semiregular<Tag>`

(23.2) — `movable-value<Data>`

(23.3) — `(sender<Child> &&...)`

(23.4) — `dependent_sender<Sndr> || sender_in<Sndr>`, where `Sndr` is *basic-sender*`<Tag, Data, Child...>` as defined below.

Recommended practice: When this mandate fails because `get_completion_signatures<Sndr>()` would exit with an exception, implementations are encouraged to include information about the exception in the resulting diagnostic.

24 *Returns*: A prvalue of type *basic-sender*`<Tag, decay_t<Data>, decay_t<Child>...>` that has been direct-list-initialized with the forwarded arguments, where *basic-sender* is the following exposition-only class template except as noted below.

```
namespace std::execution {
  template<class Tag>
  concept completion-tag = // exposition only
    same_as<Tag, set_value_t> || same_as<Tag, set_error_t>
        || same_as<Tag, set_stopped_t>;

  template<template<class...> class T, class... Args>
    concept  valid-specialization  =  requires  {  typename
        T<Args...>; }; // exposition only

  struct default-impls {  // exposition only
    static constexpr auto get-attrs = see below;
    static constexpr auto get-env = see below;
    static constexpr auto get-state = see below;
    static constexpr auto start = see below;
    static constexpr auto complete = see below;
```

```
    template<class Sndr, class... Env>
    static constexpr void check-types();
  };

  … as before …

  template <class Sndr>
    using  data-type  =  decltype(declval<Sndr>().template
        get<1>());       // exposition only

  template <class Sndr, size_t I = 0>
   using  child-type  =  decltype(declval<Sndr>().template
        get<I+2>());     // exposition only

  … as before …

  template<class Sndr, class... Env>
     using  completion-signatures-for  =  see  below;  //
        exposition only

  template<class Tag, class Data, class... Child>
  struct basic-sender : product-type<Tag, Data, Child...>
        {  // exposition only
    using sender_concept = sender_t;
     using indices-for = index_sequence_for<Child...>; //
        exposition only

    decltype(auto) get_env() const noexcept {
      auto& [_, data, ...child] = *this;
      return impls-for<Tag>::get-attrs(data, child...);
    }

    template<decays-to<basic-sender> Self, receiver Rcvr>
    auto connect(this Self&& self, Rcvr rcvr) noexcept(see
        below)
      -> basic-operation<Self, Rcvr> {
      return {std::forward<Self>(self), std::move(rcvr)};
    }

    template<decays-to<basic-sender> Self, class... Env>
      static  constexpr  auto  get_completion_signatures();
        (this Self&& self, Env&&... env) noexcept
      -> completion-signatures-for<Self, Env...> {
      return {};
    }
  };
}
```

[Editor's note: In [exec.snd.expos], replace para 39 with the paragraphs shown below and renumber subsequent paragraphs:]

39 Let `Sndr` be a (possibly `const`-qualified) specialization *`basic-sender`* or an lvalue reference of such, let `Rcvr` be the type of a receiver with an associated environment of type `Env`. If the type *`basic-operation`*`<Sndr, Rcvr>` is well-formed, let `op` be an lvalue subexpression of that type. Then *`completion-signatures-for`*`<Sndr, Env>` denotes a specialization of `completion_signatures`, the set of whose template arguments corresponds to the set of completion operations that are

potentially evaluated ([basic.def.odr]) as a result of evaluating `op.start()`. Otherwise, `completion-signatures-for<Sndr, Env>` is ill-formed. If `completion-signatures-for<Sndr, Env>` is well-formed and its type is not dependent upon the type `Env`, `completion-signatures-for<Sndr>` is well-formed and denotes the same type; otherwise, `completion-signatures-for<Sndr>` is ill-formed.

```
template <class Sndr, class... Env>
  static constexpr void default-impls::check-types();
```

? Let `Is` be the pack of integral template arguments of the `integer_sequence` specialization denoted by `indices-for<Sndr>`.

? *Effects*: Equivalent to:

```
(get_completion_signatures<child-type<Sndr, Is>, FWD-ENV-
        T(Env)...>(), ...)
```

? *Remarks*: For any types `T`, `S`, and pack `E`, let `e` be the expression `impls-for<T>::check-types<S, E...>()`. Then exactly one of the following is `true`:

(?.1) — `e` is ill-formed, or

(?.3) — The evaluation of `e` exits with an exception, or

(?.2) — `e` is a core constant expression.

When `e` is a core constant expression, the types `S`, `E...` uniquely determine a set of completion signatures.

```
template<class Tag, class Data, class... Child>
  template <class Sndr, class... Env>
                    constexpr     auto     basic-sender<Tag,     Data,
        Child...>::get_completion_signatures();
```

? Let `Rcvr` be the type of a receiver whose environment has type `E`, where `E` is the first type in the list `Env..., env<>`. Let `CHECK-TYPES()` be the expression `impls-for<Tag>::template check-types<Sndr, E>()`, and let `CS` be a type determined as follows:

(?.1) — If `CHECK-TYPES()` is a core constant expression, let `op` be an lvalue subexpression whose type is `connect_result_t<Sndr, Rcvr>`. Then `CS` is the specialization of `completion_signatures` the set of whose template arguments correspond to the set of completion operations that are potentially evaluated ([basic.def.odr]) as a result of evaluating `op.start()`.

(?.2) — Otherwise, `CS` is `completion_signatures<>`.

? *Constraints*: `CHECK-TYPES()` is a well-formed expression.

? *Effects*: Equivalent to

```
CHECK-TYPES();
return CS();
```

[Editor's note: Change the specification of *write-env* in [exec.snd.expos] para 40-43 as follows:]

```
template<sender Sndr, queryable Env>
  constexpr auto write-env(Sndr&& sndr, Env&& env);          // exposition only
```

40 *write-env* is an exposition-only sender adaptor that, when connected with a receiver `rcvr`, connects the adapted sender with a receiver whose execution environment is the result of joining the queryable argument `env` to the result of `get_env(rcvr)`.

41 Let *write-env-t* be an exposition-only empty class type.

42 *Returns*:

```
make-sender(write-env-t(), std::forward<Env>(env), std::forward<Sndr>(sndr))
```

43 *Remarks*: The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for *write-env-t* as follows:

```
template<>
struct impls-for<write-env-t> : default-impls {
  static constexpr auto join-env(const auto& state, const auto& env) noexcept {
    return see below;
  }

  static constexpr auto get-env =
      [](auto, const auto& state, const auto& rcvr) noexcept {
          return join-env(state, FWD-ENV(get_env(rcvr)));
    };

  template<class Sndr, class... Env>
  static constexpr void check-types();
};
```

Invocation of *impls-for*<*write-env-t*>::~~*get-env*~~*join-env* returns an object `e` such that

(43.1) — `decltype(e)` models `queryable` and

(43.2) — given a query object `q`, the expression `e.query(q)` is expression-equivalent to `state.query(q)` if that expression is valid, otherwise, `e.query(q)` is expression-equivalent to ~~`get_env(rcvr)`~~`env.query(q)`.

(43.3) — For type `Sndr` and pack `Env`, let `State` be *data-type*`<Sndr>` and let `JoinEnv` be the pack `decltype(`*join-env*`(declval<State>(),` *FWD-ENV*`(declval<Env>())))`. Then *impls-for*<*write-env-t*>::*check-types*`<Sndr, Env...>()` is expression-equivalent to `get_completion_signatures<`*child-type*`<Sndr>, JoinEnv...>()`.

[Editor's note: Add the following new paragraphs to the end of [exec.snd.expos]]

?
```
template<class... Fns>
struct overload-set : Fns... {
  using Fns::operator()...;
};
```

[Editor's note: The following is moved from [exec.on] para 6 and modified.]

?
```
struct not-a-sender {
  using sender_concept = sender_t;

  template<class Sndr>
    static constexpr auto get_completion_signatures() -> completion_signatures<> {
    throw unspecified;
  }
};
```

?
```
constexpr void decay-copyable-result-datums(auto cs) {
  cs.for-each([]<class Tag, class... Ts>(Tag(*)(Ts...)) {
    if constexpr (!(is_constructible_v<decay_t<Ts>, Ts> &&...))
      throw unspecified;
  });
}
```

[Editor's note: Change [exec.snd.concepts] para 1 and add a new para after 1 as follows:]

1 The `sender` concept *… as before …* to produce an operation state.

```
namespace std::execution {
  template<class Sigs>
    concept valid-completion-signatures = see below;        // exposition only

  template<auto>
    concept is-constant = true;        // exposition only

  template<class Sndr>
    concept is-sender =        // exposition only
      derived_from<typename Sndr::sender_concept, sender_t>;

  template<class Sndr>
    concept enable-sender =        // exposition only
      is-sender<Sndr> ||
      is-awaitable<Sndr, env-promise<env<>>>;        // [exec.awaitable]

  template<class Sndr>
    consteval bool is-dependent-sender-helper() try {        // exposition only
      get_completion_signatures<Sndr>();
      return false;
    } catch (dependent-sender-error&) {
      return true;
```

```
    }

  template<class Sndr>
    concept sender =
      bool(enable-sender<remove_cvref_t<Sndr>>) &&
      requires (const remove_cvref_t<Sndr>& sndr) {
        { get_env(sndr) } -> queryable;
      } &&
      move_constructible<remove_cvref_t<Sndr>> &&
      constructible_from<remove_cvref_t<Sndr>, Sndr>;

  template<class Sndr, class... Env>
    concept sender_in =
      sender<Sndr> &&
      (queryable<Env> &&...) &&
       is-constant<get_completion_signatures<Sndr, Env...>
         ()>
      requires (Sndr&& sndr, Env&&... env) {
            { get_completion_signatures(std::forward<Sndr>
         (sndr), std::forward<Env>(env)...) }
           -> valid-completion-signatures;
      };

  template<class Sndr>
    concept dependent_sender =
       sender<Sndr> && bool_constant<is-dependent-sender-
         helper<Sndr>()>::value;

  template<class Sndr, class Rcvr>
    concept sender_to =
      sender_in<Sndr, env_of_t<Rcvr>> &&
        receiver_of<Rcvr, completion_signatures_of_t<Sndr,
         env_of_t<Rcvr>>> &&
      requires (Sndr&& sndr, Rcvr&& rcvr) {
                            connect(std::forward<Sndr>(sndr),
         std::forward<Rcvr>(rcvr));
      };
}
```

? — For a type `Sndr`, if `sender<Sndr>` is `true` and `dependent_sender<Sndr>` is `false`, then `Sndr` is a non-dependent sender ([exec.async.ops]).

[Editor's note: Strike [exec.snd.concepts] para 3 (this para is moved to [execution.syn]):]

3 ~~A type models the exposition-only concept *`valid-completion-signatures`* if it denotes a specialization of the `completion_signatures` class template.~~

[Editor's note: Change [exec.getcomplsigs] as follows:]

1 `get_completion_signatures` is a customization point object. Let `sndr` be an expression such that `decltype((sndr))` is `Sndr`, and let `env` be a pack of zero or one expression. If `sizeof...(env) == 0` is `true`, let `new_sndr` be `sndr`; otherwise, let `new_sndr` be the expression `transform_sender(decltype(get-domain-late(sndr, env...)){}, sndr, env...)`. Let `NewSndr` be `decltype((new_sndr))`. Then `get_completion_signatures(sndr, env...)` is

expression-equivalent to `(void(sndr), void(env)..., CS())` except that `void(sndr)` and `void(env)...` are indeterminately sequenced, where `CS` is:

(1.1) — `decltype(new_sndr.get_completion_signatures(env...))` if that type is well-formed,

(1.2) — Otherwise, if `sizeof...(env) == 1` is `true`, then `decltype(new_sndr.get_completion_signatures())` if that expression is well-formed,

(1.3) — Otherwise, `remove_cvref_t<NewSndr>::completion_signatures` if that type is well-formed,

(1.4) — Otherwise, if `is-awaitable<NewSndr, env-promise<decltype((env))>...>` is `true`, then:

```
completion_signatures<
   SET-VALUE-SIG(await-result-type<NewSndr, env-promise<Env>>),
        //  ([exec.snd.concepts])
  set_error_t(exception_ptr),
  set_stopped_t()>
```

(1.4) — Otherwise, `CS` is ill-formed.

```
template <class Sndr, class... Env>
   consteval auto get_completion_signatures() -> valid-completion-
        signatures auto;
```

? Let *`except`* be an rvalue subexpression of an unspecified class type *`Except`* such that `move_constructible<Except> && derived_from<Except, exception>` is `true`. Let *`CHECKED-COMPLSIGS`*`(`*`e`*`)` be *`e`* if *`e`* is a core constant expression whose type satisfies *`valid-completion-signatures`*; otherwise, it is the following expression:

```
(e, throw except, completion_signatures())
```

Let *`get-complsigs`*`<Sndr, Env...>()` be expression-equivalent to `remove_reference_t<Sndr>::template get_completion_signatures<Sndr, Env...>()`, let *`nested-complsigs-t`*`<Sndr>` be an alias for the type `typename remove_reference_t<Sndr>::completion_signatures`, and let `NewSndr` be `Sndr` if `sizeof...(Env) == 0` is `true`; otherwise, `decltype(`*`s`*`)` where *`s`* is the following expression:

```
transform_sender(
  get-domain-late(declval<Sndr>(), declval<Env>()...),
  declval<Sndr>(),
  declval<Env>()...)
```

? *Constraints*: `sizeof...(Env) <= 1` is `true`.

? *Effects*: Equivalent to: `return` *`e`*`;` where *`e`* is expression-equivalent to the following:

(?.1) — *CHECKED-COMPLSIGS*(*get-complsigs*<NewSndr, Env...>()) if *get-complsigs*<NewSndr, Env...>() is a well-formed expression.

(?.2) — Otherwise, *CHECKED-COMPLSIGS*(*get-complsigs*<NewSndr>()) if *get-complsigs*<NewSndr>() is a well-formed expression.

(?.3) — Otherwise, *CHECKED-COMPLSIGS*(*nested-complsigs-t*<NewSndr>()) if *nested-complsigs-t*<NewSndr>() is a well-formed expression.

(?.4) — Otherwise,

```
completion_signatures<
      SET-VALUE-SIG(await-result-type<NewSndr,   env-
        promise<Env>...>),  //  ([exec.snd.concepts])
  set_error_t(exception_ptr),
  set_stopped_t()>
```

if `is-awaitable<NewSndr, env-promise<Env>...>` is `true`.

(?.5) — Otherwise, `(throw dependent-sender-error(), completion_signatures())` if `sizeof...(Env) == 0` is `true`,

(?.6) — Otherwise, `(throw except, completion_signatures())`.

2 [Editor's note: This para is no longer needed because the new `dependent_sender` concept covers it.] ~~If `get_completion_signatures(sndr)` is well-formed and its type denotes a specialization of the `completion_signatures` class template, then `Sndr` is a non-dependent sender type ([exec.async.ops]).~~

3 Given a type `Env`, if `completion_signatures_of_t<Sndr>` and `completion_signatures_of_t<Sndr, Env>` are both well-formed, they shall denote the same type.

4 Let `rcvr` be an rvalue whose type `Rcvr` models `receiver`, and let `Sndr` be the type of a sender such that `sender_in<Sndr, env_of_t<Rcvr>>` is `true`. Let `Sigs...` be the template arguments of the `completion_signatures` specialization named by `completion_signatures_of_t<Sndr, env_of_t<Rcvr>>`. Let `CSO` be a completion function. If sender `Sndr` or its operation state cause the expression `CSO(rcvr, args...)` to be potentially evaluated ([basic.def.odr]) then there shall be a signature `Sig` in `Sigs...` such that

```
MATCHING-SIG(decayed-typeof<CSO>(decltype(args)...), Sig)
```

is `true` ([exec.general]).

[Editor's note: At the very bottom of [exec.connect], change the *Mandates* of para 6 as follows:]

6 The expression `connect(sndr, rcvr)` is expression-equivalent to:

(6.1) — `new_sndr.connect(rcvr)` if that expression is well-formed.

Mandates: The type of the expression above satisfies `operation_state`.

(6.2) — Otherwise, *`connect-awaitable`*`(new_sndr, rcvr)`.

Mandates: ~~`sender<Sndr> && receiver<Rcvr>`~~ The following are `true`:

(6.3) — `sender_in<Sndr, env_of_t<Rcvr>>`

(6.4) — `receiver_of<Rcvr, completion_signatures_of_t<Sndr, env_of_t<Rcvr>>>`

[Editor's note: In [exec.read.env] para 3, make the following change:]

3 The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for `read_env` as follows:

```
namespace std::execution {
  template<>
   struct  impls-for<decayed-typeof<read_env>>  :  default-
        impls {
    static constexpr auto start =
      [](auto query, auto& rcvr) noexcept -> void {
                                       TRY-SET-VALUE(std::move(rcvr),
        query(get_env(rcvr)));
      };

    template<class Sndr, class Env>
    static constexpr void check-types();
  };
}
```

```
template<class Sndr, class Env>
static constexpr void check-types();
```

(3.1) — Let `Q` be `decay_t<`*`data-type`*`<Sndr>>`.

(3.2) — *Throws*: An exception of an unspecified type derived from `exception` if the expression `Q()(env)` is ill-formed or has type `void`, where `env` is an lvalue subexpression whose type is `Env`.

[Editor's note: Change [exec.schedule.from] para 4 and insert a new para between 6 and 7 as follows:]

4 The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for `schedule_from_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<schedule_from_t> : default-impls {
    static constexpr auto get-attrs = see below;
    static constexpr auto get-state = see below;
    static constexpr auto complete = see below;

    template<class Sndr, class... Env>
    static constexpr void check-types();
```

```
  };
}
```

5 The member … as before …

6 The member *impls-for*<schedule_from_t>::*get-state* is initialized with a callable object equivalent to the following lambda: [Editor's note: This integrates the resolution from LWG#4203.]

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)
        noexcept(see below)
          requires sender_in<child-type<Sndr>, FWD-ENV-T(env_of_t<Rcvr>)> {

  auto& [_, sch, child] = sndr;
  … as before …
```

```
template<class Sndr, class... Env>
static constexpr void check-types();
```

? *Effects*: Equivalent to:

```
get_completion_signatures<schedule_result_t<data-
        type<Sndr>>, FWD-ENV-T(Env)...>();
auto cs = get_completion_signatures<child-type<Sndr>, FWD-
        ENV-T(Env)...>();
decay-copyable-result-datums(cs); // see [exec.snd.expos]
```

7 Objects of the local class *state-type* … as before …

8 Let Sigs be a pack of the arguments to the completion_signatures specialization named by completion_signatures_of_t<*child-type*<Sndr>, *FWD-ENV-T*(env_of_t<Rcvr>)>. Let *as-tuple* be an alias template such that *as-tuple*<Tag(Args...)> denotes the type *decayed-tuple*<Tag, Args...>. Then variant_t denotes the type variant<monostate, *as-tuple*<Sigs>...>, except with duplicate types removed.

[Editor's note: Change [exec.on] para 6 as follows (*not-a-sender* is moved to [exec.snd.expos]):]

6 Otherwise: Let *not-a-scheduler* be an unspecified empty class type.~~, and let *not-a-sender* be the exposition-only type:~~

```
struct not-a-sender {
  using sender_concept = sender_t;

  auto get_completion_signatures(auto&&) const {
    return see below;
  }
};
```

~~where the member function get_completion_signatures returns an object of a type that is not a specialization of the completion_signatures class template.~~

[Editor's note: Delete [exec.on] para 9 as follows:]

9 ~~*Recommended practice*: Implementations should use the return type of *not-a-sender*::get_completion_signatures to inform users that their usage of on is incorrect because there is no available scheduler onto which to restore execution.~~

[Editor's note: Revert the change to [exec.then] made by P3164R3, and then change [exec.then] para 4 as follows:]

4 The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for *then-cpo* as follows:

```
namespace std::execution {
  template<>
   struct  impls-for<decayed-typeof<then-cpo>>  :  default-
        impls {
    static constexpr auto complete =
      []<class Tag, class... Args>
        (auto, auto& fn, auto& rcvr, Tag, Args&&... args)
        noexcept -> void {
           if constexpr (same_as<Tag, decayed-typeof<set-
        cpo>>) {
              TRY-SET-VALUE(rcvr,
                                              invoke(std::move(fn),
        std::forward<Args>(args)...));
          } else {
                Tag()(std::move(rcvr), std::forward<Args>
        (args)...);
          }
        };

    template<class Sndr, class... Env>
    static constexpr void check-types();
  };
}
```

?
```
template<class Sndr, class... Env>
static constexpr void check-types();
```

(?.1) — *Effects:* Equivalent to:

```
auto  cs  =  get_completion_signatures<child-type<Sndr>,  FWD-ENV-
        T(Env)...>();
auto fn = []<class... Ts>(set_value_t(*)(Ts...)) {
    if  constexpr  (!invocable<remove_cvref_t<data-type<Sndr>>,
        Ts...>)
    throw unspecified;
};
cs.for-each(overload-set{fn, [](auto){}});
```

[Editor's note: Revert the change to [exec.let] made by P3164R3, and then change [exec.let] para 5 and insert a new para after 5 as follows:]

5 The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for *let-cpo* as follows:

```
namespace std::execution {
  template<class State, class Rcvr, class... Args>
```

```
  void let-bind(State& state, Rcvr& rcvr, Args&&... args);
        // exposition only

  template<>
    struct impls-for<decayed-typeof<let-cpo>> : default-
        impls {
      static constexpr auto get-state = see below;
      static constexpr auto complete = see below;

      template<class Sndr, class... Env>
      static constexpr void check-types();
  };
}
```

6 Let *receiver2* denote the following exposition-only class template:

```
namespace std::execution {
  … as before …
}
```

Invocation of the function *receiver2*::get_env returns an object e such that

(6.1) — decltype(e) models *queryable* and

(6.2) — given a query object q, the expression e.query(q) is expression-equivalent to *env*.query(q) if that expression is valid~~,~~; otherwise, if the type of q satisfies *forwarding-query*, e.query(q) is expression-equivalent to get_env(*rcvr*).query(q); otherwise, e.query(q) is ill-formed.

?
```
template<class Sndr, class... Env>
static constexpr void check-types();
```

(?.1) — *Effects:* Equivalent to:

```
using LetFn = remove_cvref_t<data-type<Sndr>>;
auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-
        T(Env)...>();
auto fn = []<class... Ts>(decayed-typeof<set-cpo>(*)(Ts...)) {
  if constexpr (!is-valid-let-sender)
    throw unspecified;
};
cs.for-each(overload-set(fn, [](auto){}));
```

where *is-valid-let-sender* is true if and only if all of the following are true:

(?.1.1) — `(constructible_from<decay_t<Ts>, Ts> &&...)`

(?.1.2) — `invocable<LetFn, decay_t<Ts>&...>`

(?.1.3) — `sender<invoke_result_t<LetFn, decay_t<Ts>&...>>`

(?.1.4) — `sizeof...(Env) == 0 || sender_in<invoke_result_t<LetFn, decay_t<Ts>&...>, env-t...>`

where *env-t* is the pack `decltype(let-cpo.transform_env(declval<Sndr>(), declval<Env>()))`.

[Editor's note: The following changes are the proposed resolutions to cplusplus/sender-receiver#316 and cplusplus/sender-receiver#318.]

7 *`impls-for`*`<`*`decayed-typeof`*`<`*`let-cpo`*`>>::`*`get-state`* is initialized with a callable object equivalent to the following:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)
        requires see below {
  auto& [_, fn, child] = sndr;
  using fn_t = decay_t<decltype(fn)>;
  using env_t = decltype(let-env(child));
  using args_variant_t = see below;
  using ops2_variant_t = see below;
  … as before …
}
```

(7.1) — Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<`*`child-type`*`<Sndr>,` *`FWD-ENV-T`*`(env_of_t<Rcvr>)>`. Let `LetSigs` be a pack of those types in `Sigs` with a return type of *`decayed-typeof`*`<`*`set-cpo`*`>`. Let *`as-tuple`* be an alias template such that *`as-tuple`*`<Tag(Args...)>` denotes the type *`decayed-tuple`*`<Args...>`. Then `args_variant_t` denotes the type `variant<monostate,` *`as-tuple`*`<LetSigs>...>` except with duplicate types removed.

(7.2) — Given a type `Tag` and a pack `Args`, let *`as-sndr2`* be an alias template such that *`as-sndr2`*`<Tag(Args...)>` denotes the type *`call-result-t`*`<Fn, decay_t<Args>&...>`. Then `ops2_variant_t` denotes the type

```
variant<monostate, connect_result_t<as-sndr2<LetSigs>, receiver2<Rcvr, ~~Env~~env_t>>...>
```

except with duplicate types removed.

(7.3) — The *`requires-clause`* constraining the above lambda is satisfied if and only if the types `args_variant_t` and `ops2_variant_t` are well-formed.

11 The exposition-only function template *`let-bind`* has effects equivalent to: … as before …

12 … as before …

[Editor's note: The following change to [exec.let] para 13 is the proposed resolution to cplusplus/sender-receiver#319.]

13 Let `sndr` and `env` be subexpressions, and let `Sndr` be `decltype((sndr))`. If *`sender-for`*`<Sndr,` *`decayed-typeof`*`<`*`let-cpo`*`>>` is `false`, then the expression *`let-cpo`*`.transform_env(sndr, env)` is ill-formed. Otherwise, it is equal to ~~*`JOIN-ENV`*`(`*`let-env`*`(sndr),` *`FWD-ENV`*`(env))`.~~

```
auto& [_, _, child] = sndr;
return JOIN-ENV(let-env(child), FWD-ENV(env));
```

[Editor's note: Revert the change to [exec.bulk] made by P3164R3, and then change [exec.bulk] para 3 and insert a new para after 5 as follows:]

3 The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for `bulk_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<bulk_t> : default-impls {
    static constexpr auto complete = see below;

    template<class Sndr, class... Env>
    static constexpr void check-types();
  };
}
```

4 The member `impls-for<bulk_t>::complete` is … as before …

5 … as before …

?
```
template<class Sndr, class... Env>
static constexpr void check-types();
```

(?.1) — *Effects:* Equivalent to:

```
auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-
        T(Env)...>();
auto fn = []<class... Ts>(set_value_t(*)(Ts...)) {
    if constexpr (!invocable<remove_cvref_t<data-type<Sndr>>,
        Ts&...>)
    throw unspecified;
};
cs.for-each(overload-set{fn, [](auto){}});
```

[Editor's note: Revert the change to [exec.split] made by P3164R3, and then change [exec.split] para 3 and insert a new para after 3 as follows:]

3 The name `split` denotes a pipeable sender adaptor object. ~~For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `sender_in<Sndr, split-env>` is `false`, `split(sndr)` is ill-formed.~~

? The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for `split_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<split_t> : default-impls {
    template<class Sndr>
    static constexpr void check-types() {
          auto cs = get_completion_signatures<child-
        type<Sndr>, split-env>();
             decay-copyable-result-datums(cs);  // see
        [exec.snd.expos]
    }
  };
}
```

[Editor's note: Change [exec.when.all] paras 2-9 and insert two new paras after 4 as follows:]

2 The names `when_all` and `when_all_with_variant` denote customization point objects. Let `sndrs` be a pack of subexpressions, let `Sndrs` be a pack of the types `decltype((sndrs))...`, and let `CD` be the type `common_type_t<decltype(`*`get-domain-early`*`(sndrs))...>` , and let `CD2` be `CD` if `CD` is well-formed, and `default_domain` otherwise.. The expressions `when_all(sndrs...)` and `when_all_with_variant(sndrs...)` are ill-formed if any of the following is true:

(2.1) — `sizeof...(sndrs)` is `0`, or

(2.2) — `(sender<Sndrs> && ...)` is `false`~~, or~~.

(2.3) ~~— `CD` is ill-formed.~~

3 The expression `when_all(sndrs...)` is expression-equivalent to:

```
transform_sender(CD()CD2(),    make-sender(when_all,    {},
        sndrs...))
```

4 The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for `when_all_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<when_all_t> : default-impls {
    static constexpr auto get-attrs = see below;
    static constexpr auto get-env = see below;
    static constexpr auto get-state = see below;
    static constexpr auto start = see below;
    static constexpr auto complete = see below;

    template<class Sndr, class... Env>
    static constexpr void check-types();
  };
}
```

? Let `make-when-all-env` be the following exposition-only function template:

```
template<class Env>
constexpr    auto    make-when-all-env(inplace_stop_source&
        stop_src, Env&& env) noexcept {
  return see below;
}
```

Returns an object `e` such that *The following itemized list has been moved here from para 6 and modified.*

(?.1) — `decltype(e)` models `queryable`, and

(?.2) — `e.query(get_stop_token)` is expression-equivalent to `stop_src.get_token()`, and

(?.3) — given a query object `q` with type other than *cv* `stop_token_t` and whose type satisfies *`forwarding-query`*, `e.query(q)` is expression-equivalent to env.query(q).

Let *when-all-env* be an alias template such that *when-all-env*<Env> denotes the type decltype(*make-when-all-env*(declval<inplace_stop_source&>(), declval<Env>())).

?

```
template<class Sndr, class... Env>
static constexpr void check-types();
```

(?.1) — Let Is be the pack of integral template arguments of the integer_sequence specialization denoted by *indices-for*<Sndr>.

(?.2) — *Effects*: Equivalent to:

```
auto fn = []<class Child>() {
     auto cs = get_completion_signatures<Child, when-all-
        env<Env>...>();
  if constexpr (cs.count-of(set_value) >= 2)
    throw unspecified;
  decay-copyable-result-datums(cs); // see [exec.snd.expos]
};
(fn.template operator()<child-type<Sndr, Is>>(), ...);
```

(?.3) — *Throws*: Any exception thrown as a result of evaluating the *Effects*, or an exception of an unspecified type when CD is ill-formed.

5 The member *impls-for*<when_all_t>::*get-attrs* … as before …

6 The member *impls-for*<when_all_t>::*get-env* is initialized with a callable object equivalent to the following lambda expression:

```
[]<class State, class Rcvr>(auto&&, State& state, const
        Receiver& rcvr) noexcept {
     return ~~see below~~make-when-all-env(state.stop-src,
        get_env(rcvr));
}
```

~~Returns an object e such that~~

(6.1) — ~~decltype(e) models queryable, and~~

(6.2) — ~~e.query(get_stop_token) is expression-equivalent to state.*stop-src*.get_token(), and~~

(6.3) — ~~given a query object q with type other than *cv* stop_token_t, e.query(q) is expression-equivalent to get_env(rcvr).query(q).~~

7 The member *impls-for*<when_all_t>::*get-state* is initialized with a callable object equivalent to the following lambda expression:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)
        noexcept(e) -> decltype(e) {
  return e;
}
```

where *e* is the expression

```
std::forward<Sndr>(sndr).apply(make-state<Rcvr>())
```

and where *make-state* is the following exposition-only class template:

```
template<class Sndr, class Env>
concept   max-1-sender-in    =    sender_in<Sndr,    Env>    &&
        // exposition only@
  (tuple_size_v<value_types_of_t<Sndr, Env, tuple, tuple>>
        <= 1);

enum  class  disposition  {  started,  error,  stopped  };
        // exposition only

template<class Rcvr>
struct make-state {
  template<max-1-sender-in<env_of_t<Rcvr>>class... Sndrs>
  auto operator()(auto, auto, Sndrs&&... sndrs) const {
    using values_tuple = see below;
    using errors_variant = see below;
                                 using       stop_callback       =
        stop_callback_for_t<stop_token_of_t<env_of_t<Rcvr>>,
        on-stop-request>;
… as before …
```

8 Let *copy-fail* be *… as before …*

9 The alias `values_tuple` denotes the type

```
tuple<value_types_of_t<Sndrs,   FWD-ENV-T(env_of_t<Rcvr>),
        decayed-tuple, optional>...>
```

if that type is well-formed; otherwise, `tuple<>`.

[Editor's note: Change [exec.when.all] para 14 as follows:]

14 The expression `when_all_with_variant(sndrs...)` is expression-equivalent to:

```
transform_sender(CD()CD2(),                              make-
        sender(when_all_with_variant, {}, sndrs...));
```

[Editor's note: Change [exec.into.variant] paras 4-5 as follows (with the change to para 5 being a drive-by fix):]

4 The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for *into_variant* as follows:

```
namespace std::execution {
  template<>
  struct impls-for<into_variant_t> : default-impls {
    static constexpr auto get-state = see below;
    static constexpr auto complete = see below;

    template<class Sndr, class... Env>
    static constexpr void check-types() {
                 auto   cs   =   get_completion_signatures<child-
        type<Sndr>, FWD-ENV-T(Env)...>();
              decay-copyable-result-datums(cs);     //   see
        [exec.snd.expos]
```

```
    }
  };
}
```

5 The member *impls-for*<into_variant_t>::*get-state* is initialized with a callable object equivalent to the following lambda:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)
        noexcept
  -> type_identity<value_types_of_t<child-type<Sndr>, FWD-
        ENV-T(env_of_t<Rcvr>)>> {
  return {};
}
```

[Editor's note: Revert the change to [exec.stopped.opt] made by P3164R3, and make the following changes instead. Note: this includes the proposed resolution to cplusplus/sender-receiver#311]

2 The name `stopped_as_optional` denotes a pipeable sender adaptor object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `stopped_as_optional(sndr)` is expression-equivalent to:

```
transform_sender(get-domain-early(sndr),            make-
        sender(stopped_as_optional, {}, sndr))
```

except that `sndr` is only evaluated once.

? The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for `stopped_as_optional_t` as follows:

```
template<>
struct impls-for<stopped_as_optional_t> : default-impls {
  template<class Sndr, class... Env>
  static constexpr void check-types() {
    default-impls::check-types<Sndr, Env...>();
    if constexpr (!requires {
           requires (!same_as<void, single-sender-value-
        type<child-type<Sndr>, FWD-ENV-T(Env)...>>); })
      throw unspecified;
  }
};
```

3 Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If *sender-for*<Sndr, stopped_as_optional_t> is `false`~~, or if the type *single-sender-value-type*<Sndr, Env> is ill-formed or void,~~ then the expression `stopped_as_optional.transform_sender(sndr, env)` is ill-formed; otherwise, if `sender_in<child-type<Sndr>, FWD-ENV-T(Env)>` is `false`, the expression `stopped_as_optional.transform_sender(sndr, env)` is equivalent to *not-a-sender*(); otherwise, it is equivalent to:

```
auto&& [_, _, child] = sndr;
using  V  =  single-sender-value-type<child-type<Sndr>,  FWD-ENV-
        T(Env)>;
return let_stopped(
    then(std::forward_like<Sndr>(child),
```

```
                                    []<class...    Ts>(Ts&&...    ts)
        noexcept(is_nothrow_constructible_v<V, Ts...>) {
          return optional<V>(in_place, std::forward<Ts>(ts)...);
        }),
    []() noexcept { return just(optional<V>()); });
```

[Editor's note: Change [exec.util.cmplsig] para 8 and add a new para after 8 as follows:]

8
```
namespace std::execution {
  template<completion-signature... Fns>
    struct completion_signatures {
      template<class Tag>
      static constexpr size_t count-of(Tag) { return see below; }

      template<class Fn>
        static constexpr void for-each(Fn&& fn) { // exposition only
          (std::forward<Fn>(fn)(static_cast<Fns*>(nullptr)), ...);
        }
    };

  … as before …
}
```

? For a subexpression `tag`, let `Tag` be the decayed type of `tag`. `completion_signatures<Fns...>::count-of(tag)` returns the count of function types in `Fns...` that are of the form `Tag(Ts...)` where `Ts` is a pack of types.

[Editor's note: Remove subclause [exec.util.cmplsig.trans].]

10 Proposed Wording for the Nice-to-haves

[Editor's note: Change [execution.syn] as follows (changes relative to what is already suggested above):]

```
… as before …

namespace std::execution {
  … as before …

  // [exec.util], sender and receiver utilities
  // [exec.util.cmplsig] completion signatures
  template<class Fn>
    concept completion-signature = see below;                        //
        exposition only

  template<completion-signature... Fns>
    struct completion_signatures;

  template<class Sigs>
    concept valid-completion-signatures = see below;                 //
        exposition only

  struct dependent-sender-error : exception {};                      //
        exposition only

  // [exec.getcomplsigs]
```

```
  template<class Sndr, class... Env>
    consteval auto get_completion_signatures() -> valid-completion-signatures auto;

  template<class Sndr, class... Env>
    requires sender_in<Sndr, Env...>
      using completion_signatures_of_t = decltype(get_completion_signatures<Sndr, Env...>());

  template<bool PotentiallyThrowing>
    inline constexpr auto eptr_completion_if = completion_signatures();

  template<>
    inline constexpr auto eptr_completion_if<true> =
      completion_signatures<set_error_t(exception_ptr)>();

  // [exec.util.cmplsig.getchild]
  template <class Parent, class Child, class... Env>
    consteval auto get_child_completion_signatures() -> valid-completion-signatures auto;

  // [exec.util.cmplsig.make]
  template <completion-signature... ExplicitSigs, completion-signature... DeducedSigs>
    constexpr auto make_completion_signatures(DeducedSigs*... sigs) noexcept
      -> valid-completion-signatures auto;

  // [exec.util.cmplsig.invalid]
  template <class... Types, class... Values>
    [[noreturn, nodiscard]]
    consteval auto invalid_completion_signature(Values... values)
      -> completion_signatures<>;

  // [exec.util.cmplsig.trans]
  template<
      valid-completion-signatures Completions,
      class ValueTransform = see below,
      class ErrorTransform = see below,
      class StoppedTransform = see below,
      valid-completion-signatures ExtraCompletions = see below>
    constexpr auto transform_completion_signatures(
        Completions completions,
        ValueTransform value_transform = {},
        ErrorTransform error_transform = {},
        StoppedTransform stopped_transform = {},
        ExtraCompletions extra_completions = {});

  // [exec.run.loop], run_loop
  class run_loop;
}

… as before …
```

[Editor's note: Make the completion tags equality comparable with each other. Insert a new paragraph after [exec.rcvr.concepts] as follows:]

(33.7.?) **Completion tags [exec.set.tag]**

1 The types `set_value_t`, `set_error_t`, and `set_stopped_t` are completion tag types ([exec.async.ops]). Each models `equality_comparable` with instances comparing equal to each other. Each also models `equality_comparable_with` the other two, with objects of different types comparing not equal. For two completion tag objects `a` and `b`, `a == b` and `a != b` are core constant expressions and are not potentially throwing. [*Example:* `set_value_t() == set_value_t()` is a constant expression that evaluates to `true`, and `set_value_t() == set_error_t()` is a constant expression that evaluates to `false`. — *end example*]

[Editor's note: Give the `completion_signatures` class template some `constexpr` members for manipulating its instances. Instead of the change suggested above to [exec.util.cmplsig] para 8, split para 8 into two after `struct completion_signatures` and modify as follows:]

8
```
namespace std::execution {
  template<completion-signature... Fns>
    struct completion_signatures {
      completion_signatures() = default;

      constexpr explicit completion_signatures(Fns*...) noexcept
        requires (0 != sizeof...(Fns)) {}

        static constexpr size_t size() noexcept { return sizeof...
        (Fns); }

      template<class... Others>
        static constexpr bool
        contains(completion_signatures<Others...>) noexcept;

      template<class Fn>
        static constexpr auto filter(Fn) noexcept;

      template<class Tag>
        static constexpr auto select(Tag) noexcept;

      template<class MapFn, callable<call-result-t<MapFn, Fns*>...>
        ReduceFn>
        static constexpr auto transform_reduce(MapFn map, ReduceFn
        reduce);

      template<class... Others>
        constexpr auto operator+(completion_signatures<Others...>)
        noexcept;

      template<class... Others>
        constexpr bool operator==(completion_signatures<Others...>
        other) noexcept;
    };

  template<size_t I, class... Fns>
    constexpr auto get(completion_signatures<Fns...>) noexcept ->
        Fns...[I]* {
      return nullptr;
    }
}
```

```
namespace std {
  using execution::get;

  template<class... Fns>
    struct tuple_size<execution::completion_signatures<Fns...>>
      : integral_constant<size_t, sizeof...(Fns)> {};

  template<size_t I, class... Fns>
                                                     struct          tuple_element<I,
        execution::completion_signatures<Fns...>> {
      using type = Fns...[I]*;
    };
}
```

(8.1) — *Mandates*: For the specialization `completion_signatures<Fns...>`, the types `NORMALIZE-SIG(Fns)...` are unique.

```
template<class... Others>
constexpr bool contains(completion_signatures<Others...>)
        noexcept;
```

(8.2) — *Returns*: `true` if and only if for all types `T` in `Others...` the expression `(MATCHING-SIG(T, Fns) ||...)` is `true`.

```
template<class Fn>
constexpr auto filter(Fn) noexcept;
```

(8.3) — For a type `Ret` and pack `Args`, let `maybe-completion<Ret(Args...)>` denote the type `completion_signatures<Ret(Args...)>` if `callable<Fn, Ret(*)(Args...)>` is `true`, and `completion_signatures<>` otherwise.

(8.4) — *Returns*: `(completion_signatures() +...+ maybe-completion<Fns>())`.

```
template<class Tag>
constexpr auto select(Tag) noexcept;
```

(8.5) — *Returns*: `filter([]<class... Ts>(Tag(*)(Ts...)) {})`.

```
template<class    MapFn,    callable<call-result-t<MapFn,
        Fns*>...> ReduceFn>
constexpr   auto   transform_reduce(MapFn   map,   ReduceFn
        reduce);
```

(8.6) — *Returns*: `reduce(map(static_cast<Fns*>(nullptr))...)`.

```
template<class... Others>
constexpr auto operator+(completion_signatures<Others...>)
        noexcept;
```

(8.7) — *Returns*: `completion_signatures<Us...>()`, where `Us` is the pack of the types in `NORMALIZE-SIG(Fns)...` and `NORMALIZE-SIG(Others)...` with duplicate types removed.

```
template<class... Others>
constexpr bool operator==(completion_signatures<Others...>
        other) const noexcept;
```

(8.7) — *Returns*: `size() == other.size() && contains(other)`.

[Editor's note: The following is split from what is currently para 8 in the Working Draft.]

9
```
namespace std::execution {
  template<class Sndr, class Env = env<>,
          template<class...> class Tuple = decayed-tuple,
          template<class...> class Variant = variant-or-empty>
      requires sender_in<Sndr, Env>
    using value_types_of_t =
                                          gather-signatures<set_value_t,
        completion_signatures_of_t<Sndr, Env>, Tuple, Variant>;

  template<class Sndr, class Env = env<>,
          template<class...> class Variant = variant-or-empty>
      requires sender_in<Sndr, Env>
    using error_types_of_t =
                                          gather-signatures<set_error_t,
        completion_signatures_of_t<Sndr, Env>,
                        type_identity_t, Variant>;

  template<class Sndr, class Env = env<>>
      requires sender_in<Sndr, Env>
    constexpr bool sends_stopped =
      !same_as<type-list<>,
                                        gather-signatures<set_stopped_t,
        completion_signatures_of_t<Sndr, Env>,
                                  type-list, type-list>>;

}
```

[Editor's note: After [exec.util.cmplsig], insert a three new sections as follows:]

(33.10.?) `execution::get_child_completion_signatures` [exec.util.cmplsig.getchild]

```
template <class Parent, class Child, class... Env>
    consteval auto get_child_completion_signatures() -> valid-completion-signatures auto;
```

? *Effects*: Equivalent to:

```
using CvChild = decltype(std::forward_like<Parent>(declval<Child&>()));
return get_completion_signatures<CvChild, FWD-ENV-T(Env)...>();
```

(33.10.?) `execution::make_completion_signatures` [exec.util.cmplsig.make]

```
template <completion-signature... ExplicitSigs, completion-signature... DeducedSigs>
    constexpr auto make_completion_signatures(DeducedSigs*... sigs) noexcept
  -> valid-completion-signatures auto;
```

1 — *Returns*: `completion_signatures<Ts...>()`, where `Ts` is a pack of the types *`NORMALIZE-SIG`*`(ExplicitSigs)...`, *`NORMALIZE-SIG`*`(DeducedSigs)...` with duplicate types removed.

(33.10.?) `execution::invalid_completion_signature` [exec.util.cmplsig.invalid]

```
template <class... Types, class... Values>
  [[noreturn, nodiscard]]
    consteval  auto  invalid_completion_signature(Values...
         values)
     -> completion_signatures<>;
```

1 — `invalid_completion_signature` is used to report type errors encountered while computing a sender's completion signatures.

2 — [*Example 1*: A `current_scheduler` algorithm might use `invalid_completion_signature` in its sender type as follows:

```
template <const auto&> struct IN_ALGORITHM;
extern const struct current_scheduler_t current_scheduler; // an
        algorithm

template <class Sndr, class Env>
constexpr                                                    auto
        current_scheduler_sender::get_completion_signatures() {
  if constexpr (invocable<get_scheduler_t, Env>) {
    using Result = invoke_result_t<get_scheduler_t, Env>;
    return completion_signatures<set_value_t(Result)>();
  } else {
    // Oh no, the user made an error! Tell them about it.
    return invalid_completion_signature<
      IN_ALGORITHM<current_scheduler>,
      struct NO_SCHEDULER_IN_THE_CURRENT_ENVIRONMENT,
      struct WITH_ENVIRONMENT(Env)
    >("The current execution environment does not have a value "
      "for the get_scheduler query.");
  }
}
```

– *end example*]

3 — *Effects*: Equivalent to `return (throw` *`unspecified`*`, completion_signatures());`.

4 — *Throws*: An exception of an unspecified type derived from `exception`.

5 — *Recommended practice*: Implementations are encouraged to throw an exception such that the template parameters and function arguments of `invalid_completion_signature` appear in the compiler diagnostic should the exception propagate out of a `consteval` context.

[Editor's note: Replace the section [exec.util.cmplsig.trans] in the Working Draft with the following:]

(33.10.2) `execution::transform_completion_signatures` [exec.util.cmplsig.trans]

1 `transform_completion_signatures` is ~~an alias~~ a function template used to transform one set of completion signatures into another. It takes a set of completion signatures and several other ~~template~~ arguments that apply

modifications to each completion signature in the set to generate an instance of a new specialization of `completion_signatures`.

2 [*Example 1*: Given a sender type `Sndr` and an environment `Env`, adapt the completion signatures of `Sndr` by lvalue-ref qualifying the values, adding an additional `exception_ptr` error completion if it is not already there, and leaving the other completion signatures alone.

```
template<class... Args>
  using my_set_value_t =
    completion_signatures<
      set_value_t(add_lvalue_reference_t<Args>...)>;

usingauto my_completion_signatures =
  transform_completion_signatures<(
    get_completion_signatures_of_t<Sndr, Env>(),
     []<class... Args>() { return my_set_value_t<Args...>
        (); },
    {},   // no-op error signature transform
    {},   // no-op stopped signature transform
    completion_signatures<set_error_t(exception_ptr)>());,
    my_set_value_t>;
```

– *end example*]

[Editor's note: Replace the remaining paragraphs of [exec.util.cmplsig.trans] with the following:]

3 This subclause makes use of the following exposition-only entities:

```
template<class Tag>
  constexpr auto pass-thru-transform = []<class... Ts>()
        noexcept {
    return completion_signatures<Tag(Ts...)>();
  };

constexpr auto concat-completions = [](auto... cs)
        noexcept {
  return (completion_signatures() +...+ cs);
};
```

(3.1) — For a subexpression `fn` and a pack of types `Args`, let *`TRANSFORM-EXPR`*`(fn, Args)` be expression-equivalent to `fn()` if `sizeof...(Args) == 0` is `true` and `fn.template operator()<Args...>()` otherwise. Let *`APPLY-TRANSFORM`*`(fn, Args)` be expression-equivalent to:

(3.1.1) — *`TRANSFORM-EXPR`*`(fn, Args)` if that expression is well-formed and has a type that satisfies *`valid-completion-signatures`*.

(3.1.2) — Otherwise, `(`*`TRANSFORM-EXPR`*`(fn, Args), throw `*`unspecified`*`, completion_signatures())` if that expression is well-formed.

(3.1.3) — Otherwise, `(throw `*`unspecified`*`, completion_signatures())`.

4

```
template<
   valid-completion-signatures Completions,
            class    ValueTransform    =    decltype(pass-thru-
      transform<set_value_t>),
            class    ErrorTransform    =    decltype(pass-thru-
      transform<set_error_t>),
           class    StoppedTransform    =    decltype(pass-thru-
      transform<set_stopped_t>),
           valid-completion-signatures    ExtraCompletions    =
      completion_signatures<>>
 constexpr auto transform_completion_signatures(
     Completions completions,
     ValueTransform value_transform = {},
     ErrorTransform error_transform = {},
     StoppedTransform stopped_transform = {},
     ExtraCompletions extra_completions = {});
```

(4.1) — *Effects*: Equivalent to

```
auto transform = [=]<class Tag, class... Args>(Tag(*)
        (Args...)) {
  if constexpr (same_as<Tag, set_value_t>)
    return APPLY-TRANSFORM(value_transform, Args);
  else if constexpr (same_as<Tag, set_error_t>)
    return APPLY-TRANSFORM(error_transform, Args);
  else
    return APPLY-TRANSFORM(stopped_transform, Args);
};

return        completions.transform_reduce(transform,
        concat-completions)
  + extra_completions;
```

(4.2) — *Recommended practice*: Users are encouraged to throw descriptive exceptions from their transformation functions when they encounter errors during type computation.

§ 11 Acknowledgements

I would like to thank Hana Dusíková for her work making constexpr exceptions a reality for C++26. Thanks are also due to David Sankel for his encouragement to investigate using constexpr exceptions as an alternative to TMP hackery, and for giving feedback on an early draft of this paper.

§ 12 References

[P2830R7] Gašper Ažman, Nathan Nichols. 2024-11-21. Standardized Constexpr Type Ordering.
https://wg21.link/p2830r7
[P3068R6] Hana Dusíková. 2024-11-19. Allowing exception throwing in constant-evaluation.
https://wg21.link/p3068r6
[P3164R2] Eric Niebler. 2024-06-25. Improving diagnostics for sender expressions.
https://wg21.link/p3164r2

[P3164R3] Eric Niebler. 2025-01-10. Early Diagnostics for Sender Expressions.
https://wg21.link/p3164r3

---

1. https://godbolt.org/z/xGMMrrnYa↩

2. https://gist.github.com/ericniebler/0896776ab1c8f5b7f77d7094c0400df5↩